

net-05 - Geneve Overlay com Open vSwitch

Contexto [↗](#)

Neste treinamento iremos abordar:

- Funcionamento do GENEVE com OVS
- Hands on - Criando conectividade entre workloads de diferentes hosts usando GENEVE
- Hands on - Analisando os headers dos pacotes encapsulados com GENEVE
- Hands on - Desafio

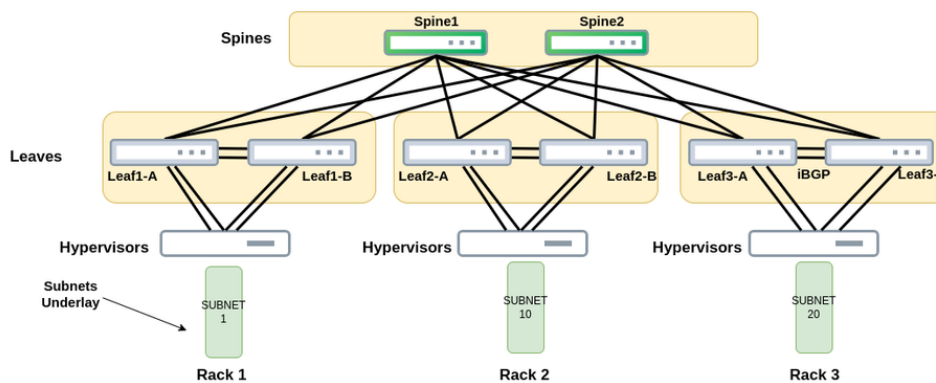
Funcionamento do GENEVE com OVS [↗](#)

Antes de iniciar o treinamento, no caso de você não saber o que é GENEVE ou protocolos de encapsulamento, sugiro antes de prosseguir iniciar com o treinamento 04

Nesse treinamento vamos focar na parte prática de como programar o OVS para usar o GENEVE na criação de túneis entre diferentes hosts para criar conectividade entre workloads mas sem envolver uma SDN Controller por enquanto.

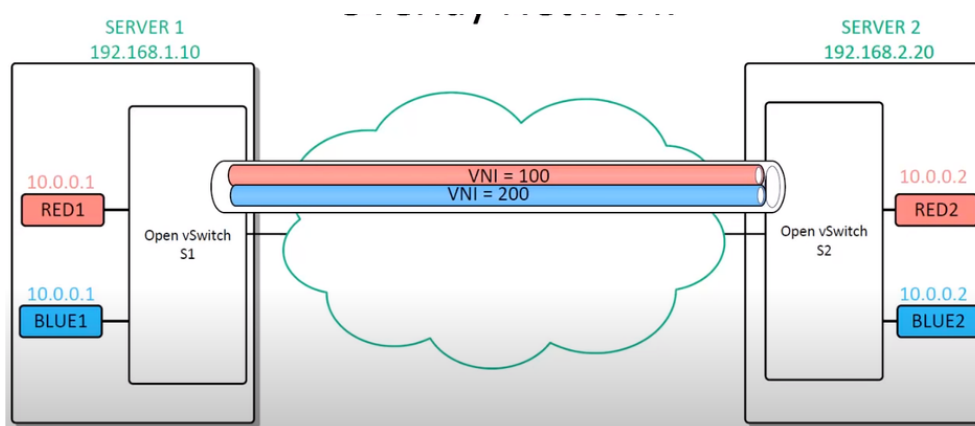
Antes de mais nada, temos que frisar que os protocolos de encapsulamento criam túneis entre hypervisors independente da arquitetura de rede do Underlay.

Em uma arquitetura de rede de Underlay tradicional Spine-Leaf, os hypervisors são distribuídos em inúmeros racks e são conectados na camada de Leaf que possui roteamento via Spine que permite criar roteamento com qualquer outro rack.



Do ponto de vista do Overlay, os workloads podem estar dispostos em diferentes hypervisors, em diferentes racks e podendo ter conectividade em uma rede virtual L2 criada pelo Overlay por intermédio de um protocolo de encapsulamento.

Com o GENEVE seria possível criar conectividade L2 entre os workloads em uma rede virtual que é identificada por um VNI (Virtual Network Identifier) conforme abaixo:



O OVS (Open vSwitch) possui alguns tipos de interfaces que usamos:

- **Normal** - é um tipo de interface que possibilita montar uma NIC física do OS do host no OVS
 - ex: A eth1 é uma interface de rede física do host e abaixo é criado uma associação entre a porta OVS br-ext com a eth1

```
1 # ovs-vsctl add-port br-ext eth1
```

- **Internal** - é um tipo de interface virtual criada pelo OVS utilizada para conectar aos workloads
 - ex: A interface virtual tap10 será associada na bridge br-int

```
1 # ovs-vsctl add-port br-int tap10 -- set Interface tap10
2 # ovs-vsctl set Interface tap10 type=internal
3 # ip a
4 ...
5 5: tap10: <BROADCAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN qlen 1000
6     link/ether 5a:d7:49:85:d9:da brd ff:ff:ff:ff:ff:ff
7     inet6 fe80::58d7:49ff:fe85:d9da/64 scope link
8         valid_lft forever preferred_lft forever
```

- **Patch** - quando existem diferentes bridges gerenciadas pelo OVS, é possível criar conectividade entre elas usando a Patch Port. O tráfego de rede recebido por uma patch port de uma bridge será encaminhado para a outra bridge via patch port.
 - ex:

```
1 # ovs-vsctl add-br br0
2 # ovs-vsctl add-br br1
3 # ovs-vsctl \
4 -- add-port br0 patch0 -- set interface patch0 type=patch options:peer=patch1 \
5 -- add-port br1 patch1 -- set interface patch1 type=patch options:peer=patch0
```

- **Tunnel** - tunneling é utilizado para criar uma rede virtual em cima do Underlay e possibilitando o tráfego de rede L2. Tunneling permite que bridges em diferentes hosts possam ter conectividade como se estivessem no mesmo host.
 - ex:

```
1 -(Host A) Tunnel entre Host A e Host B com VNI 100
2 # ovs-vsctl add-br br-int
3 # ovs-vsctl add-port br-int tun0 -- set Interface tun0 type=geneve options:key=100 options:remote_ip=192.168.7.23
4 # ovs-vsctl show
5 12c25989-892b-4eaa-ad82-bbccc02f8383
6     Bridge br-int
7         Port br-int
8             Interface br-int
9                 type: internal
10        Port tun0
11            Interface tun0
12                type: geneve
13                options: {key="100", remote_ip="192.168.7.23"}
14        ovs_version: "2.17.0"
15
16 -(Host B) Tunnel entre Host B e Host A com VNI 100
17 # ovs-vsctl add-br br-int
18 # ovs-vsctl add-port br-int tun0 -- set Interface tun0 type=geneve options:key=100 options:remote_ip=192.168.6.10
19 # ovs-vsctl show
20 a8551f03-0b71-4fd4-9cf1-9f8fe0dc7d21
21     Bridge br-int
22         Port tun0
23             Interface tun0
24                 type: geneve
25                 options: {key="100", remote_ip="192.168.6.10"}
26        Port br-int
27            Interface br-int
28                type: internal
29        ovs_version: "2.17.0"
```

Criando conectividade entre workloads de diferentes hosts usando GENEVE [↗](#)

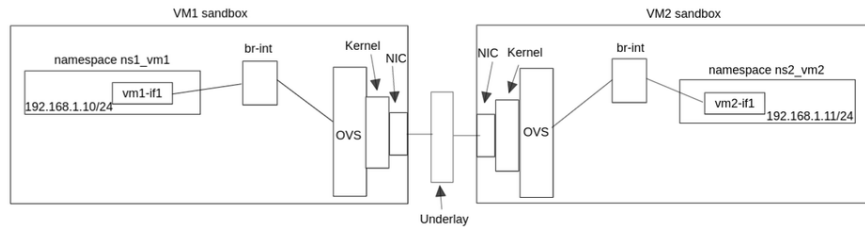
i Para fins de teste **apenas**, é necessário ter 2 sandboxes VMs na mesma subnet executando o Ubuntu 20.04 com os pacotes latest instalados para seguir os exercícios.

1 - Instalando o OVS via pacote em ambas VMs [↗](#)

Por favor, siga os passos do treinamento 01 para a instalação do **OVS 2.17**.

2 - Criando conectividade entre os workloads [↗](#)

Neste exercício, faremos a criação do setup abaixo:



Após ter finalizado a instalação do OVS em ambas as VMs, execute a seguinte sequência de comandos para criar nosso cenário:

i Neste exemplo a VM1 possui o IP 10.0.10.100 e a VM2 o 10.0.10.200, altere os valores usados abaixo conforme seu setup

#VM1

```
1 # ovs-vsctl add-br br-int
2 # ovs-vsctl add-port br-int tun0 -- set Interface tun0 type=geneve options:key=100 options:remote_ip=10.0.10.200
3 # ip netns add ns1_vm1
4 # ovs-vsctl add-port br-int vm1-if1 -- set interface vm1-if1 type=internal
5 # ip link set vm1-if1 netns ns1_vm1
6 # ip netns exec ns1_vm1 ip addr add 192.168.1.10/24 dev vm1-if1
7 # ip netns exec ns1_vm1 ip link set vm1-if1 up
```

#VM2

```
1 # ovs-vsctl add-br br-int
2 # ovs-vsctl add-port br-int tun0 -- set Interface tun0 type=geneve options:key=100 options:remote_ip=10.0.10.100
3 # ip netns add ns2_vm2
4 # ovs-vsctl add-port br-int vm2-if1 -- set interface vm2-if1 type=internal
5 # ip link set vm2-if1 netns ns2_vm2
6 # ip netns exec ns2_vm2 ip addr add 192.168.1.11/24 dev vm2-if1
7 # ip netns exec ns2_vm2 ip link set vm2-if1 up
```

Nesse momento já é possível analisar se o namespace **ns1_vm1** consegue ter conectividade com o namespace **ns2_vm2**.

```
1 # ip netns exec ns1_vm1 ping -c 3 192.168.1.11
2 PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data.
3 64 bytes from 192.168.1.11: icmp_seq=1 ttl=64 time=0.513 ms
4 64 bytes from 192.168.1.11: icmp_seq=2 ttl=64 time=0.459 ms
5 64 bytes from 192.168.1.11: icmp_seq=3 ttl=64 time=0.486 ms
6
7 --- 192.168.1.11 ping statistics ---
8 3 packets transmitted, 3 received, 0% packet loss, time 2049ms
9 rtt min/avg/max/mdev = 0.459/0.486/0.513/0.022 ms
```

Essa comunicação só é possível pois o **ovs-vsitchd** instalou automaticamente o seguinte flow com a action NORMAL em ambos os lados:

```
1 # ovs-ofctl dump-flows br-int
2 cookie=0x0, duration=1814.529s, table=0, n_packets=2322, n_bytes=219236, priority=0 actions=NORMAL
```

Verifique que temos um túnel GENEVE configurado em ambas as VMs sandbox:

#VM1

```
1 # ovs-vsctl show
2 a8551f03-0b71-4fd4-9cf1-9f8fe0dc7d21
3 Bridge br-int
4 Port tun0
5 Interface tun0
6 type: geneve
7 options: {key="100", remote_ip="10.0.10.200"}
8 Port br-int
9 Interface br-int
10 type: internal
11 Port vm1-if1
12 Interface vm1-if1
13 type: internal
14 ovs_version: "2.17.0"
```

#VM2

```
1 # ovs-vsctl show
2 12c25989-892b-4eaa-ad82-bbccc02f8383
3 Bridge br-int
4 Port br-int
5 Interface br-int
```

```

6         type: internal
7     Port tun0
8         Interface tun0
9             type: geneve
10            options: {key="100", remote_ip="10.0.10.100"}
11     Port vm2-if1
12         Interface vm2-if1
13             type: internal
14     ovs_version: "2.17.0"

```

Com a ajuda do **ovs-ofctl**, veremos quais portas estão disponíveis:

```

1 # ovs-ofctl show br-int
2 OFPT_FEATURES_REPLY (xid=0x2): dpid:0000be292fa0ab42
3 n_tables:254, n_buffers:0
4 capabilities: FLOW_STATS TABLE_STATS PORT_STATS QUEUE_STATS ARP_MATCH_IP
5 actions: output enqueue set_vlan_vid set_vlan_pcp strip_vlan mod_dl_src mod_dl_dst mod_nw_src mod_nw_dst mod_nw_tos mod_tp_src mod_tp_dst
6 1(tun0): addr:76:af:48:e5:85:f3
7     config:      0
8     state:       0
9     speed: 0 Mbps now, 0 Mbps max
10 2(vm1-if1): addr:56:d4:53:0c:6b:2b
11     config:      0
12     state:       0
13     speed: 0 Mbps now, 0 Mbps max
14 LOCAL(br-int): addr:be:29:2f:a0:ab:42
15     config:      PORT_DOWN
16     state:       LINK_DOWN
17     speed: 0 Mbps now, 0 Mbps max
18 OFPT_GET_CONFIG_REPLY (xid=0x4): frags=normal miss_send_len=0

```

No output acima, a interface **tun0** cria o túnel geneve entre os hosts.

Vamos analisar como o **Datapath** irá se comportar ao realizarmos o trace de comunicação entre os namespaces. Execute os seguintes comandos conforme a indicação da VM sandbox e salve o MAC Address:

#VM1

```
1 # ovs-ofctl show br-int | grep vm1-if1
```

#VM2

```
1 # ovs-ofctl show br-int | grep vm2-if1
```

Neste exemplo os MAC Addresses dos namespaces coletados foram:

MAC Address do namespace **ns1_vm1** = 56:d4:53:0c:6b:2b

MAC Address do namespace **ns2_vm2** = 1a:9a:52:1c:0a:d8

Execute o trace abaixo na sandbox VM1 utilizando o MAC Address de destino do namespace **ns2_vm2**:

```

1 # ovs-appctl ofproto/trace br-int in_port=2,d1_dst=1a:9a:52:1c:0a:d8
2 Flow: in_port=2,vlan_tci=0x0000,d1_src=00:00:00:00:00:00,d1_dst=1a:9a:52:1c:0a:d8,d1_type=0x0000
3
4 bridge("br-int")
5 -----
6 0. priority 0
7     NORMAL
8     -> no learned MAC for destination, flooding
9     -> output to kernel tunnel
10
11 Final flow: unchanged
12 MegafLOW: recirc_id=0,eth,in_port=2,d1_src=00:00:00:00:00:00,d1_dst=1a:9a:52:1c:0a:d8,d1_type=0x0000
13 Datapath actions: 1,set(tunnel(tun_id=0x64,dst=10.0.10.200,ttl=64,tp_dst=6081,flags(df|key))),2

```

No output acima, podemos determinar que o pacote será enviado via túnel (**output to kernel tunnel**), em **Datapath actions** o VNI utilizado será o 100 (0x64 em hexadecimal) com destino ao host 10.0.10.200 na porta UDP 6081(GENEVE).

Faça o trace abaixo na sandbox VM2 utilizando o MAC Address de destino do namespace **ns1_vm1**:

```

1 # ovs-appctl ofproto/trace br-int in_port=2,d1_dst=56:d4:53:0c:6b:2b
2 Flow: in_port=2,vlan_tci=0x0000,d1_src=00:00:00:00:00:00,d1_dst=56:d4:53:0c:6b:2b,d1_type=0x0000
3
4 bridge("br-int")
5 -----
6 0. priority 0
7     NORMAL
8     -> no learned MAC for destination, flooding

```

```

9      -> output to kernel tunnel
10
11 Final flow: unchanged
12 Megaflow: recirc_id=0,eth,in_port=2,d1_src=00:00:00:00:00:00,d1_dst=56:d4:53:0c:6b:2b,d1_type=0x0000
13 Datapath actions: 1,set(tunnel(tun_id=0x64,dst=10.0.10.100,ttl=64,tp_dst=6081,flags(df|key))),2

```

O mesmo comportamento pode ser observado, com o pacote sendo enviado via túnel.

Analisando os headers dos pacotes encapsulados com GENEVE [↗](#)

O próximo exercício é a continuação do anterior e vamos analisar o tráfego de rede tunelado em GENEVE entre os hosts.

Faremos o monitoramento do tráfego de rede utilizando o tool **tcpdump**, caso não o tenha instalado execute o seguinte:

```
1 # apt install -y tcpdump
```

Em ambas sandboxes VMs verifique as interfaces de rede que possuem os IPs utilizados para tunneling:

```

1 # ip a
2 ...
3 3: enp1s0.10@enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
4     link/ether 52:54:00:9e:c3:f8 brd ff:ff:ff:ff:ff:ff
5     inet 10.0.10.200/24 brd 10.0.10.255 scope global enp1s0.10
6         valid_lft forever preferred_lft forever
7     inet6 fe80::5054:ff:fe9e:c3f8/64 scope link
8         valid_lft forever preferred_lft forever

```

No output acima, a interface de rede é a enp1s0.10.

A partir da sandbox VM1, execute um ping contínuo entre o namespace **ns1_vm1** e o namespace **ns2_vm2**:

#VM1

```
1 # ip netns exec ns1_vm1 ping 192.168.1.11
```

Com acesso a sandbox VM2, execute o seguinte comando com tcpdump(altere a interface de rede conforme seu setup):

#VM2

```
1 # tcpdump -nnvi enp1s0.10 not port 22
```

O tcpdump irá capturar os pacotes que chegarem na interface de rede ignorando pacotes SSH.

A captura dos pacotes irá incluir pacotes GENEVE conforme abaixo:

```

1 18:51:02.896388 IP (tos 0x0, ttl 64, id 2513, offset 0, flags [DF], proto UDP (17), length 134)
2     10.0.10.100.48183 > 10.0.10.200.6081: Geneve, Flags [none], vni 0x64
3     IP (tos 0x0, ttl 64, id 6065, offset 0, flags [DF], proto ICMP (1), length 84)
4     192.168.1.10 > 192.168.1.11: ICMP echo request, id 2350, seq 53, length 64

```

Um pacote pode ter diversos headers, vamos deixar de lado os headers de camada 1/2 e vamos entender alguns deles abaixo:

- **Header 1** - o header indica que esse pacote é do tipo UDP
- **Header 2** - o segundo indica a comunicação entre o IP 10.0.10.100(sandbox VM1) e 10.0.10.200(sandbox VM2) na porta 6081/VNI 100 do túnel GENEVE
- **Header 3** - esse é o pacote do tipo ICMP que está sendo encapsulado pelo GENEVE
- **Header 4** - mostra o ICMP echo request do namespace **ns1_vm1** para o **ns2_vm2**

Conforme o funcionamento do ICMP, após o echo request haverá o echo reply:

```

1 18:51:02.896510 IP (tos 0x0, ttl 64, id 64259, offset 0, flags [DF], proto UDP (17), length 134)
2     10.0.10.200.1841 > 10.0.10.100.6081: Geneve, Flags [none], vni 0x64
3     IP (tos 0x0, ttl 64, id 54781, offset 0, flags [none], proto ICMP (1), length 84)
4     192.168.1.11 > 192.168.1.10: ICMP echo reply, id 2350, seq 53, length 64

```

Também é possível simplificar a visualização do output dos pacotes capturados alterando opções no tcpdump:

```

1 # tcpdump -nni enp1s0.10 not port 22
2 tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
3 listening on enp1s0.10, link-type EN10MB (Ethernet), capture size 262144 bytes
4 19:10:39.380096 IP 10.0.10.100.48183 > 10.0.10.200.6081: Geneve, Flags [none], vni 0x64: IP 192.168.1.10 > 192.168.1.11: ICMP echo request, id 2350, s
5 19:10:39.380228 IP 10.0.10.200.1841 > 10.0.10.100.6081: Geneve, Flags [none], vni 0x64: IP 192.168.1.11 > 192.168.1.10: ICMP echo reply, id 2350, seq
6 19:10:40.404078 IP 10.0.10.100.48183 > 10.0.10.200.6081: Geneve, Flags [none], vni 0x64: IP 192.168.1.10 > 192.168.1.11: ICMP echo request, id 2350, s
7 19:10:40.404212 IP 10.0.10.200.1841 > 10.0.10.100.6081: Geneve, Flags [none], vni 0x64: IP 192.168.1.11 > 192.168.1.10: ICMP echo reply, id 2350, seq
8 19:10:41.428060 IP 10.0.10.100.48183 > 10.0.10.200.6081: Geneve, Flags [none], vni 0x64: IP 192.168.1.10 > 192.168.1.11: ICMP echo request, id 2350, s
9 19:10:41.428195 IP 10.0.10.200.1841 > 10.0.10.100.6081: Geneve, Flags [none], vni 0x64: IP 192.168.1.11 > 192.168.1.10: ICMP echo reply, id 2350, seq
10 19:10:42.452101 IP 10.0.10.100.48183 > 10.0.10.200.6081: Geneve, Flags [none], vni 0x64: IP 192.168.1.10 > 192.168.1.11: ICMP echo request, id 2350, s
11 19:10:42.452243 IP 10.0.10.200.1841 > 10.0.10.100.6081: Geneve, Flags [none], vni 0x64: IP 192.168.1.11 > 192.168.1.10: ICMP echo reply, id 2350, seq

```

Desafio

Após ter aprendido sobre o funcionamento do GENEVE com o OVS, tente resolver o seguinte desafio:

- 1 - Utilize o mesmo cenário desse treinamento com 2 sandboxes VMs
- 2 - Adicione dois novos namespaces (ns3_vm3 na sandbox VM1 e ns4_vm4 na sandbox VM2)
- 3 - Crie a conectividade de rede necessária dos namespaces com o OVS em cada sandbox VM
- 4 - Estabeleça o túnel GENEVE utilizando o VNI 200
- 5 - Teste a conectividade entre os novos namespaces utilizando ICMP e analise o tráfego de rede

Conclusão

Neste treinamento foi possível entender em mais detalhes o funcionamento do GENEVE com o OVS e como estabelecer um túnel GENEVE.

Até o momento todo o processo de gerenciamento do OVS foi manual, a partir do próximo treinamento iremos introduzir a SDN Controller com o OVN para executar toda a automação de instalação dos flows.

Abaixo existe o passo a passo para realizar o cleanup nas sandboxes VMs utilizadas para os exercícios.

```
1 # apt remove -y openvswitch-switch openvswitch-common python3-openvswitch
2 # rm -rf /var/lib/openvswitch
3 # rm -rf /var/log/openvswitch
4 # rm -rf /etc/openvswitch
5 # rm -f /etc/default/openvswitch-switch
6 # ip netns delete ns1_vm1
7 # ip netns delete ns2_vm2
8 # ip netns delete ns3_vm3
9 # ip netns delete ns4_vm4
10 # reboot
```